

```

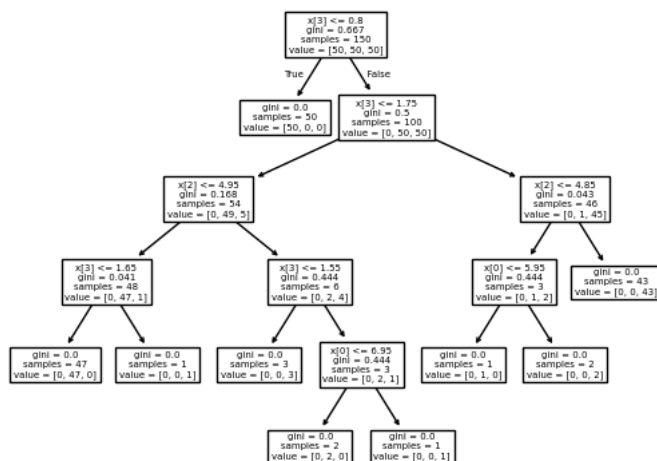
from sklearn.datasets import load_iris # примеры данных
from sklearn import tree # модуль для деревьев
iris = load_iris() # загружаем данные
X=iris.data # примеры входов
y=iris.target # метки (примеры выходов)
clf = tree.DecisionTreeClassifier() # создаем классификатор на основе дерева
clf = clf.fit(X, y) # обучаем его, т.е. создаем само дерево
tree.plot_tree(clf) # отображаем

```

```

[Text(0.5, 0.9166666666666666, 'x[3] <= 0.8\ngini = 0.667\nsamples = 150\nvalue = [50, 50, 50]'),
Text(0.4230769230769231, 0.75, 'gini = 0.0\nsamples = 50\nvalue = [50, 0, 0]'),
Text(0.46153846153846156, 0.8333333333333333, 'True '),
Text(0.5769230769230769, 0.75, 'x[3] <= 1.75\ngini = 0.5\nsamples = 100\nvalue = [0, 50, 50]'),
Text(0.5384615384615384, 0.8333333333333333, ' False'),
Text(0.3076923076923077, 0.5833333333333334, 'x[2] <= 4.95\ngini = 0.168\nsamples = 54\nvalue = [0, 49, 5]'),
Text(0.15384615384615385, 0.4166666666666667, 'x[3] <= 1.65\ngini = 0.041\nsamples = 48\nvalue = [0, 47, 1]'),
Text(0.07692307692307693, 0.25, 'gini = 0.0\nsamples = 47\nvalue = [0, 47, 0]'),
Text(0.23076923076923078, 0.25, 'gini = 0.0\nsamples = 1\nvalue = [0, 0, 1]'),
Text(0.46153846153846156, 0.4166666666666667, 'x[3] <= 1.55\ngini = 0.444\nsamples = 6\nvalue = [0, 2, 4]'),
Text(0.38461538461538464, 0.25, 'gini = 0.0\nsamples = 3\nvalue = [0, 0, 3]'),
Text(0.5384615384615384, 0.25, 'x[0] <= 6.95\ngini = 0.444\nsamples = 3\nvalue = [0, 2, 1]'),
Text(0.46153846153846156, 0.08333333333333333, 'gini = 0.0\nsamples = 2\nvalue = [0, 2, 0]'),
Text(0.6153846153846154, 0.08333333333333333, 'gini = 0.0\nsamples = 1\nvalue = [0, 0, 1]'),
Text(0.8461538461538461, 0.5833333333333334, 'x[2] <= 4.85\ngini = 0.043\nsamples = 46\nvalue = [0, 1, 45]'),
Text(0.7692307692307693, 0.4166666666666667, 'x[0] <= 5.95\ngini = 0.444\nsamples = 3\nvalue = [0, 1, 2]'),
Text(0.6923076923076923, 0.25, 'gini = 0.0\nsamples = 1\nvalue = [0, 1, 0]'),
Text(0.8461538461538461, 0.25, 'gini = 0.0\nsamples = 2\nvalue = [0, 0, 2]'),
Text(0.9230769230769231, 0.4166666666666667, 'gini = 0.0\nsamples = 43\nvalue = [0, 0, 43]')]

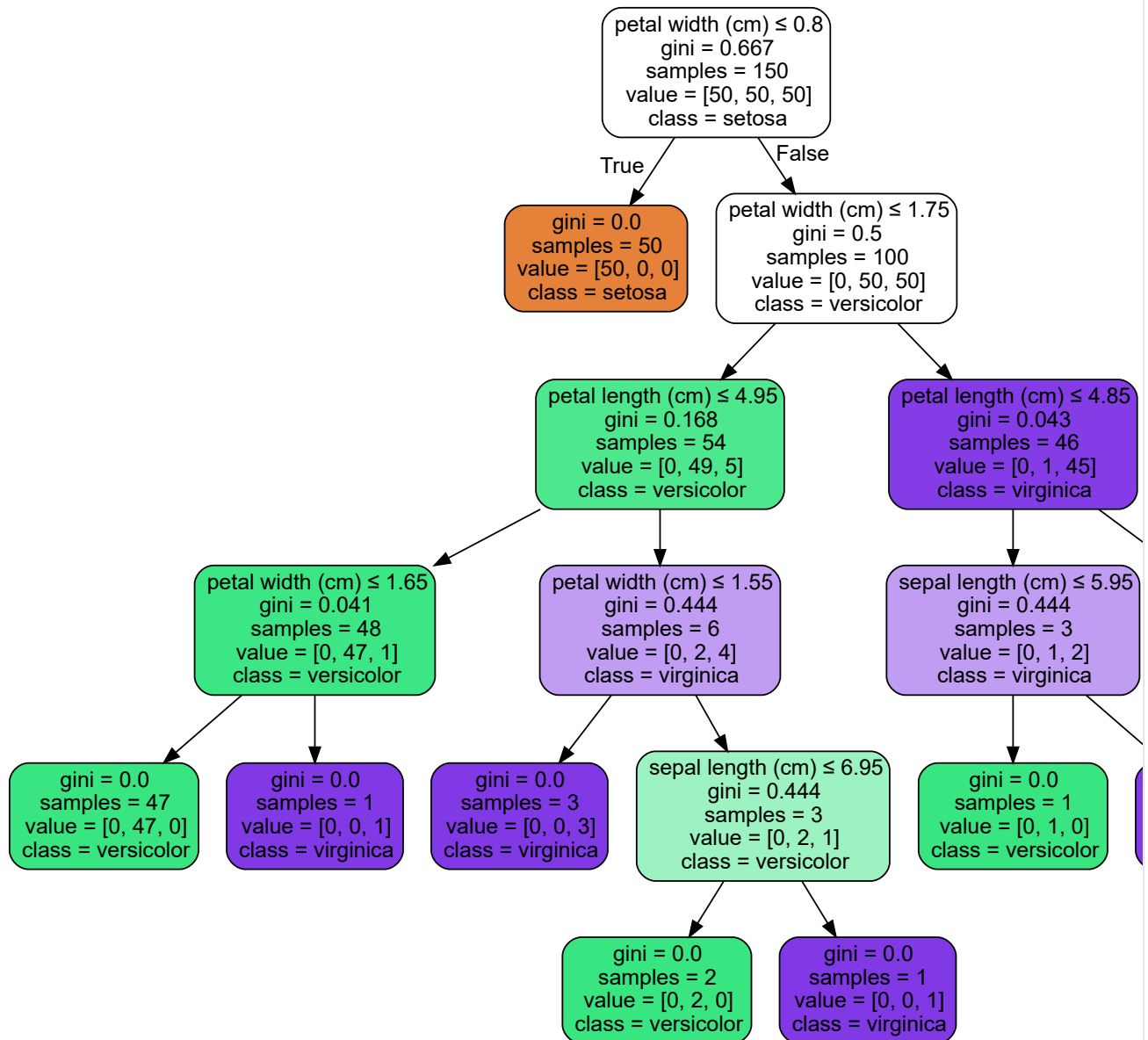
```



```

import graphviz # подключаем библиотеку для отображения деревьев
dot_data = tree.export_graphviz(clf, out_file=None) # конвертируем в DOT (в переменную или в файл)
graph = graphviz.Source(dot_data) # загружаем дерево из переменной или файла в представление graphviz
graph.render("iris") # отображаем в файл (по умолчанию pdf)
dot_data = tree.export_graphviz(clf, out_file=None, # можем указать дополнительные опции конвертации
                                feature_names=iris.feature_names, # названия атрибутов
                                class_names=iris.target_names, # названия классов
                                filled=True, rounded=True, # прочие детали отображения
                                special_characters=True) #
graph = graphviz.Source(dot_data) # # загружаем дерево из переменной или файла в представление graphviz
graph # отображаем на экране

```



```

from sklearn.tree import export_text # подключаем функцию
r = export_text(clf, feature_names=iris['feature_names']) # переводим дерево в текстовую строку
print(r) # печатаем

```

```

|--- petal width (cm) <= 0.80
|   |--- class: 0
|--- petal width (cm) > 0.80
|   |--- petal width (cm) <= 1.75
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- petal width (cm) <= 1.65
|   |   |   |   |--- class: 1
|   |   |   |   |--- petal width (cm) > 1.65
|   |   |   |   |--- class: 2
|   |   |   |--- petal length (cm) > 4.95
|   |   |   |   |--- petal width (cm) <= 1.55
|   |   |   |   |--- class: 2
|   |   |   |   |--- petal width (cm) > 1.55
|   |   |   |   |   |--- sepal length (cm) <= 6.95
|   |   |   |   |   |--- class: 1
|   |   |   |   |   |--- sepal length (cm) > 6.95
|   |   |   |   |   |--- class: 2
|   |   |--- petal width (cm) > 1.75
|   |   |   |--- petal length (cm) <= 4.85
|   |   |   |   |--- sepal length (cm) <= 5.95
|   |   |   |   |--- class: 1
|   |   |   |   |--- sepal length (cm) > 5.95
|   |   |   |   |--- class: 2
|   |   |   |--- petal length (cm) > 4.85
|   |   |   |   |--- class: 2

```

```

import numpy as np #
import matplotlib.pyplot as plt #
from sklearn.datasets import load_iris #

```

```

from sklearn.tree import DecisionTreeClassifier, plot_tree #
# Parameters
n_classes = 3 # число классов
plot_colors = "ryb" # цвета
plot_step = 0.02 # шаг для симуляции на плоскости
# Load data
iris = load_iris() # загружаем данные
# в цикле по числу пар атрибутов
for pairidx, pair in enumerate([[0, 1], [0, 2], [0, 3],
                                [1, 2], [1, 3], [2, 3]]):
    X = iris.data[:, pair] # Выбираем заданные пары атрибутов
    y = iris.target # метки
    # Обучение
    clf = DecisionTreeClassifier(max_depth=2).fit(X, y) # обучаем дерево на текущей паре атрибутов
    # Строим графики
    plt.subplot(2, 3, pairidx + 1) # подграфик для текущей пары
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1 # немного измененные минимальное и максимальные значения
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1 # немного измененные минимальное и максимальные значения
    # считаем прямоугольную сетку возможных значений этих атрибутов
    xx, yy = np.meshgrid(np.arange(x_min, x_max, plot_step), #
                          np.arange(y_min, y_max, plot_step)) #
    plt.tight_layout(h_pad=0.5, w_pad=0.5, pad=2.5) # более компактный график
    # считаем выход классификатора для всех примеров сетки
    # не забыв что массивы данных нужно привести к требуемому размеру.
    Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape) # и преобразуем обратно к исходному размеру
    cs = plt.contourf(xx, yy, Z, cmap=plt.cm.RdYlBu) # рисуем контурную карту
    plt.xlabel(iris.feature_names[pair[0]]) # добавляем подписи осей
    plt.ylabel(iris.feature_names[pair[1]]) #
    # Отображаем обучающие примеры своим цветом
    for i, color in zip(range(n_classes), plot_colors): # в цикле по количеству классов
        idx = np.where(y == i) # отбираем все точки текущего класса
        # отображаем их своим цветом
        plt.scatter(X[idx, 0], X[idx, 1], c=color, label=iris.target_names[i], #
                    cmap=plt.cm.RdYlBu, edgecolor='black', s=15) #
    plt.suptitle("Decision surface of a decision tree using paired features") # подписываем график
    plt.legend(loc='lower right', borderpad=0, handletextpad=0) # легенда
    plt.axis("tight"); # отображение осей
# построим и отобразим дерево для всех 3 классов.
plt.figure() #
clf = DecisionTreeClassifier().fit(iris.data, iris.target) #
plot_tree(clf, filled=True) #
plt.show() #

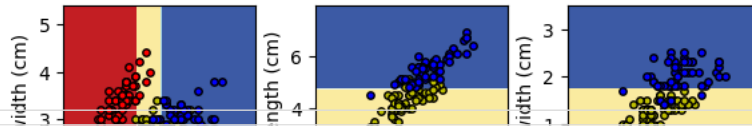
```

```

/tmp/ipython-input-4057858719.py:37: UserWarning: No data for colormapping provided via 'c'. Parameters 'cmap' will be ignore
plt.scatter(X[idx, 0], X[idx, 1], c=color, label=iris.target_names[i], #
/tmp/ipython-input-4057858719.py:37: UserWarning: No data for colormapping provided via 'c'. Parameters 'cmap' will be ignore
plt.scatter(X[idx, 0], X[idx, 1], c=color, label=iris.target_names[i], #
/tmp/ipython-input-4057858719.py:37: UserWarning: No data for colormapping provided via 'c'. Parameters 'cmap' will be ignore
plt.scatter(X[idx, 0], X[idx, 1], c=color, label=iris.target_names[i], #
/tmp/ipython-input-4057858719.py:37: UserWarning: No data for colormapping provided via 'c'. Parameters 'cmap' will be ignore
plt.scatter(X[idx, 0], X[idx, 1], c=color, label=iris.target_names[i], #
/tmp/ipython-input-4057858719.py:37: UserWarning: No data for colormapping provided via 'c'. Parameters 'cmap' will be ignore
plt.scatter(X[idx, 0], X[idx, 1], c=color, label=iris.target_names[i], #

```

Decision surface of a decision tree using paired features



clf.feature_importances_

